

Prejšnjič

- **notranji razredi**
this\$0
Reflection

```
public void izpisiHiso() throws NoSuchFieldException, SecurityException {
    Field atribut = this.getClass().getDeclaredField("this$0");
    System.out.println((Hisa)atribut.get(this));
}
```

- **primer brez uporabe upravljalca izgleda**
setLayout(null)



- **kolekcije**
vmesniki, razredi
Collections
Comparable, Comparator

```
List<Kosarkas> zvezde = new ArrayList<Kosarkas>();
zvezde.add(luka);
zvezde.add(james);
zvezde.add(luka);
```

```
public class Kosarkas implements Comparable<Kosarkas>{
    Collections.sort(zvezde);
    Collections.sort(zvezde, new PrimerjalnikNajprejVisina());
}
```

1

Collection Framework

- težave s polji

Collection	Korenski vmesnik (ne uporabljamo neposredno).
Set	Množica različnih objektov.
SortedSet	Urejena množica različnih objektov.
List	Seznam objektov
Map	Omogoča preslikavo ključa v vrednost.
SortedMap	Omogoča preslikavo urejenih ključev v vrednost.

java.util.*

- **vmesniki**: List, Set, Map
SortedSet, SortedMap, SortedSet
- **razredi**
ArrayList, TreeList, HashMap, TreeSet, TreeMap, LinkedList, HashSet
(Vector, Hashtable)
- **razred** Collections
sort(), shuffle(), reverse(),...

2

Osnovne značilnosti

- `Collection`
ni posebnega zaporedja, dovoljuje duplikate
- `List`
urejenost oz. zaporedje, dovoljuje duplikate
- `Set`
ni posebnega zaporedja, ne dovoljuje duplikatov
- `Map`
podpira iskanje po ključu, vrednost katerega mora biti enolična

3

List vs. Map

vmesnik `List`

`add(Object)`
`get(index)`

```
ArrayList<Nogometaš> nkMaribor = new ArrayList<Nogometaš>();
Nogometaš tava = new Nogometaš("Tavares", 176, 41, 36);
Nogometaš handa = new Nogometaš("Handanović", 174, 4, 11);
nkMaribor.add(tava);
nkMaribor.add(handa);
```

vmesnik `Map`

`put(ključ, vrednost)`
`get(ključ)`

```
HashMap<Integer, Nogometaš> ekipa = new HashMap<Integer, Nogometaš>();
ekipa.put(33, handa);
ekipa.put((Integer)10, zzTop);
ekipa.put(new Integer(2), miki);
ekipa.put(9, tava);
ekipa.put(11, zaho);
System.out.println(ekipa.get(9));
```

4

Operacije nad seznamami

- **Razred** `java.util.Collections`

- `static void sort(List list)`
uredi seznam po privzetih pravilih
- `static void sort(List list, Comparator c)`
uredi seznam po pravilih „komparatorja“

```
Collections.sort(nkMaribor, new PrimerjalnikStarosti());
```

- `static void shuffle(List list)`
naključno permutira seznam objektov

5

Lokalna pravila

```
imena.add("Janez");
imena.add("Ötzi");
imena.add("Đuro");

System.out.println(imena);

System.out.println("---- sortirani SLO ---");
Collections.sort(imena, Collator.getInstance(Locale.getDefault()));
// ali pa kar Collections.sort(imena, Collator.getInstance())
System.out.println(imena);

System.out.println("---- sortirani CRO ---");
Collections.sort(imena, Collator.getInstance(new Locale("hr", "HR")));
System.out.println(imena);

System.out.println("---- sortirani D ---");
Collections.sort(imena, Collator.getInstance(Locale.GERMAN));
System.out.println(imena);

---- sortirani SLO ---
[Čvetka, Janez, Janez, Karmen, Lucija, Ötzi, Špela, Vid, Žarko, Žiga, Đuro]
---- sortirani CRO ---
[Čvetka, Đuro, Janez, Janez, Karmen, Lucija, Ötzi, Špela, Vid, Žarko, Žiga]
---- sortirani D ---
[Čvetka, Janez, Janez, Karmen, Lucija, Ötzi, Špela, Vid, Žarko, Žiga, Đuro]
```

6

Urejenost

```
java.lang.Comparable <Tip>
    compareTo(drugiObjekt)
```

```
java.util.Comparator <Tip>
    compare(prvi, drugi)
```

```
java.text.Collator
    Collator.getInstance(Locale.getDefault())
    implementira vmesnik Comparator
```

7

Parametrizirani tudi vmesniki

```
import java.util.Comparator;

public class PrimerjajnikStarosti implements Comparator<Nogometaš>{

    @Override
    public int compare(Nogometaš o1, Nogometaš o2) {
        if (o1.getStarost()>o2.getStarost())
            return 1;
        else
            if (o1.getStarost()<o2.getStarost())
                return -1;
            else // sta enako mlada, uporabimo rezultat pravila privzete urejenosti
                return o1.compareTo(o2);
    }
}

public class Nogometaš implements Comparable<Nogometaš>{

    @Override
    public int compareTo(Nogometaš o) {
```

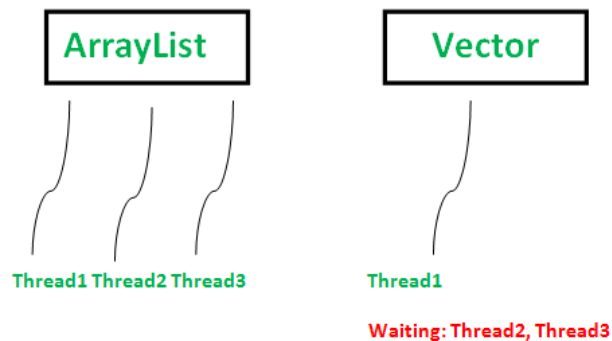
8

Nekateri **razredi**, ki implementirajo omenjene **vmesnike**

Podatkovna struktura	Set	SortedSet	List	Map	SortedMap
Sekljalna tabela	HashSet			Hashtable HashMap	
Dinamično polje	ArrayList Vector				
Uravnoreženo drevo	TreeSet			TreeMap	
Povezani seznam	LinkedList				

9

"Thread safe" kolekcije



10

Obdelava/sprehod

```

for (Nogometaš x : nkMaribor)
    System.out.println(x);

Iterator<Nogometaš> kaz = nkMaribor.iterator();
while (kaz.hasNext())
    System.out.println(kaz.next());

```

11

Parametriziran vmesniki

```

Iterator<Oseba> it = punce.iterator();
while (it.hasNext()) System.out.println(it.next());

public class PrimerjalsnikCevljev implements
    Comparator<Oseba>{

    @Override
    public int compare(Oseba prva, Oseba druga)

```

12

Ena referenca lahko predstavlja objekte različnih razredov!

```
Investicija inv = new Nepremicnina ();
```

```
inv = new VarcevalniRacun();
```

```
inv= new KriptoValuta();
```

```
Smrtonosen x = new Sosed();
```

```
Fakinski joze = new Sosed();
```

```
x = new Zmajcek();
```

13

Velja tudi za generike...

```
List<String> imena;
```

```
imena = new ArrayList<String>();
```

```
imena = new Vector<String>();
```

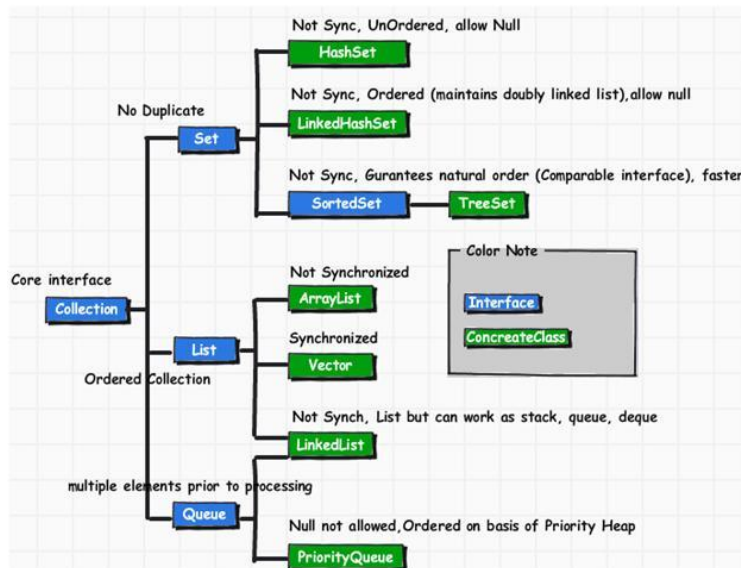
```
imena = new LinkedList<String>();
```

```
Map<Integer,String> kosarkasi;
```

```
kosarkasi = new HashMap<Integer,String>();
```

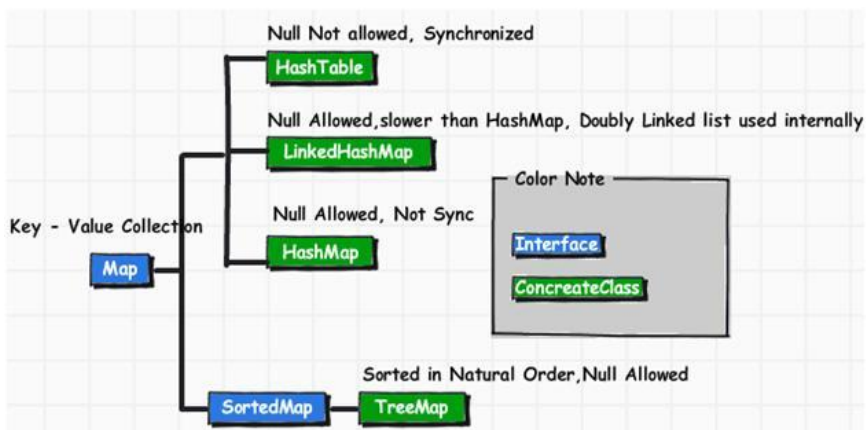
```
kosarkasi = new Hashtable<Integer,String>();
```

14



<https://www.jitendrazaa.com/blog/java/complete-java-collection-tutorial-for-the-beginner/>

15



<https://www.jitendrazaa.com/blog/java/complete-java-collection-tutorial-for-the-beginner/>

16

IO tokovi in serializacija

17

Vhodno izhodne operacije

- nivoji abstrakcije
(naprave, gonilniki, OS, program)
- knjižnica za delo z vhodno izhodnimi napravami
[java.io](https://docs.oracle.com/javase/7/docs/api/java/io/package-summary.html)

Nivoji abstrakcije

program
operacijski sistem
gonilniki
naprave

18

Kaj imamo?

```

short decimal = 100;
short octal = 0144;
short hex = 0x64;
short bin = 0b1100100;
char znak = 0b1100100;
Char znakU = '\u0064';

```

19

```

// short 16 bitov, torej 2 zloga (byte-a)

short decimal = 100;           // 1*100
short octal = 0144;            // 1*64 + 4*8 + 4
short hex = 0x64;              // 6*16 + 4
short bin = 0b0000000001100100; // 1*64 + 1*32 + 1*4
char znak = 0b0000000001100100;
char znakU = '\u0064';

System.out.println(decimal + " " + octal + " " + hex + " " + bin + " " + znak + " " + znakU);

```

100 100 100 100 d d

20

Isto zaporedje bitov

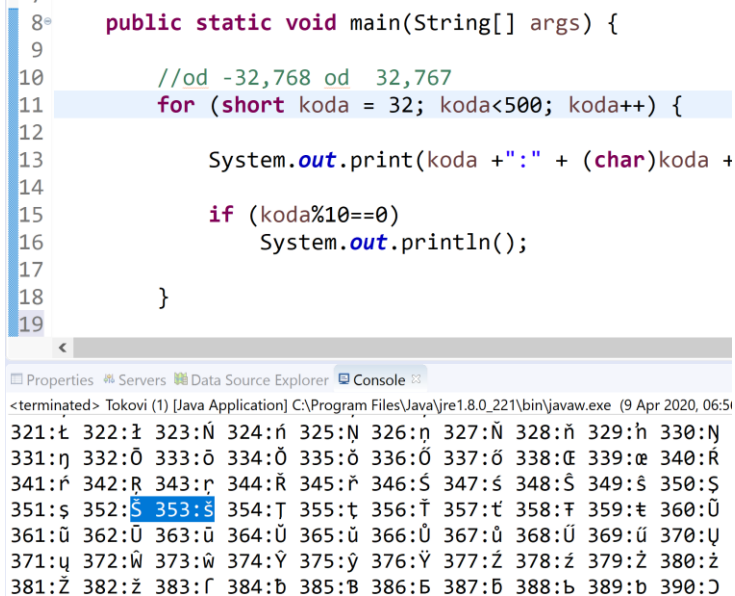
Isto zaporedje bitov – drug pomen!

```
short s = (short) 0xffff; // ti biti predstavljajo število -1
char c = '\uffff'; // isti biti predstavljajo nek Unicode znak
```

Pazimo pri pretvorbah

```
int i1 = s;           // pretvorba short v int ohrani -1
int i2 = c;           // pretvorba znaka v int 65535
```

21



```

8  public static void main(String[] args) {
9
10     //od -32,768 od 32,767
11     for (short koda = 32; koda<500; koda++) {
12
13         System.out.print(koda + ":" + (char)koda +
14
15         if (koda%10==0)
16             System.out.println();
17
18     }
19

```

<terminated> Tokovi (1) [Java Application] C:\Program Files\Java\jre1.8.0_221\bin\javaw.exe (9 Apr 2020, 06:51)

```

321:t 322:l 323:N 324:n 325:N 326:n 327:N 328:n 329:h 330:N
331:n 332:Ö 333:ö 334:Ï 335:ï 336:Ï 337:ö 338:Æ 339:æ 340:Ř
341:ř 342:Ŕ 343:ř 344:Ř 345:ř 346:Š 347:s 348:Š 349:s 350:Š
351:s 352:Š 353:š 354:T 355:t 356:Ť 357:ť 358:Ť 359:ť 360:Ů
361:ů 362:Ů 363:ů 364:Ů 365:ů 366:Ů 367:ů 368:Ů 369:ů 370:Ů
371:ů 372:Ů 373:Ů 374:Ů 375:Ů 376:Ů 377:Ů 378:Ů 379:Ů 380:Ů
381:Ž 382:ž 383:Ŧ 384:ŧ 385:Ŧ 386:ŧ 387:Ŧ 388:ŧ 389:b 390:c

```

22

Kodni nabori [uredi | uredi kodo]

Spodaj so najbolj znani **kodni nabori** in **desetiške** kode znakov (za Unicode so vrednosti **šestnajstiške**).

Standard	JUS I.B1.002			ISO 8859-2	ISO 10646
Vrsta kodiranja	YUSCII	CP852	CP1250	Latin2	Unicode
Č	94	172	200	200	U+010C
č	126	159	232	232	U+010D
Š	91	230	138	169	U+0160
š	123	231	154	185	U+0161
Ž	64	166	142	174	U+017D
ž	96	167	158	190	U+017E

https://sl.wikipedia.org/wiki/Kodiranje_%C5%A1umnikov

23

Vhodni in izhodni tokovi

```
InputStream, BufferedInputStream,
FileInputStream, DataInputStream, OutputStream,
PrintStream, FileOutputStream, DataOutputStream,
Reader, BufferedReader, FileReader, Writer,
PrintWriter, FileWriter
```

24

Binarni oz. zlogovni tokovi

byte input/output stream

- **....InputStream**

FileInputStream, ObjectInputStream, DataInputStream, BufferedInputStream itd.

- **....OutputStream**

FileOutputStream, ObjectOutputStream, DataOutputStream, BufferedOutputStream

25

Tok zlogov ali znakov

- JDK 1.1 je vpeljal podporo za tok znakov
- **znakovni** tok (character stream) je podoben kot niz zlogov (**byte stream**), le da vsebuje **16-bitne Unicode** znake in ne **8 bitne** zloge
- znakovni tokovi so implementirani s pomočjo **Reader** in **Writer** razredov in podrazredov
- podpora istim operacijam kot navadni tokovi

26

“Dvojčki”

- `java.io`
`XXXStream` razredi so zlogovno orientirani
- `XXXReader`, `XXXWriter` so znakovno orientirani
- večina funkcionalnosti zlogovnih tokov je torej zagotovljena tudi za znakovne tokove
- glej predpone, npr.
`PushbackReader` / `PushbackInputStream`

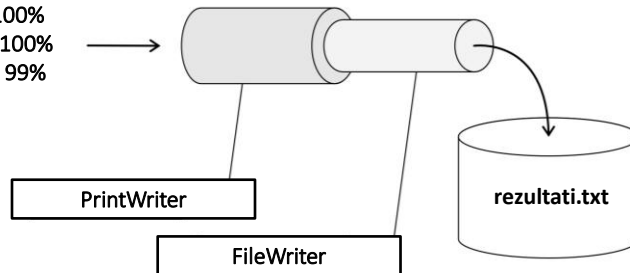
27

Zakaj znakovni tokovi

- enostavneje izdelamo programe, ki niso odvisni od določenega načina kodiranja znakov, zato je lažja **internacionalizacija**
- Java hrani nize (string-e) v **Unicode** mednarodnem formatu
- tipične uporabniške zbirke ne uporabljajo Unicode, znakovni nizi skrijejo kompleksnost različnih načinov kodiranja in sicer s pomočjo dveh posebnih razredov, ki služijo kot **most** med zlogovnimi in znakovni tokovi - `InputStreamReader` in `OutputStreamWriter`
- potencialno učinkovitejši: “buffer-at-a-time” vs. “byte-at-a-time” branje in pisanje

28

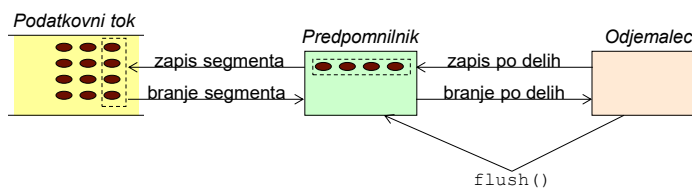
Janez - 100%
Metka - 100%
Marica - 99%



```
FileWriter pw = new FileWriter("rezultati.txt", false);
PrintWriter izhod = new PrintWriter(pw);
izhod.println("Janez - 100%");
izhod.println("Metka - 100%");
izhod.println("Marica - 99%");
izhod.flush();
izhod.close();
```

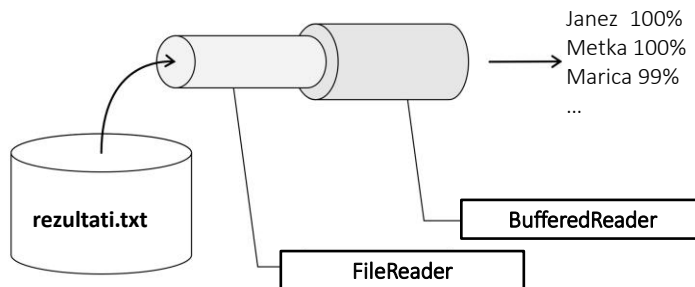
29

Predpomnilnik ("Buffer")



- Razredi, ki delajo s predpomnilnikom:
 - `BufferedInputStream` / `BufferedOutputStream`
 - `BufferedReader` / **`BufferedWriter`**
- Privzeta velikost predpomnilnika je 512 zlogov.

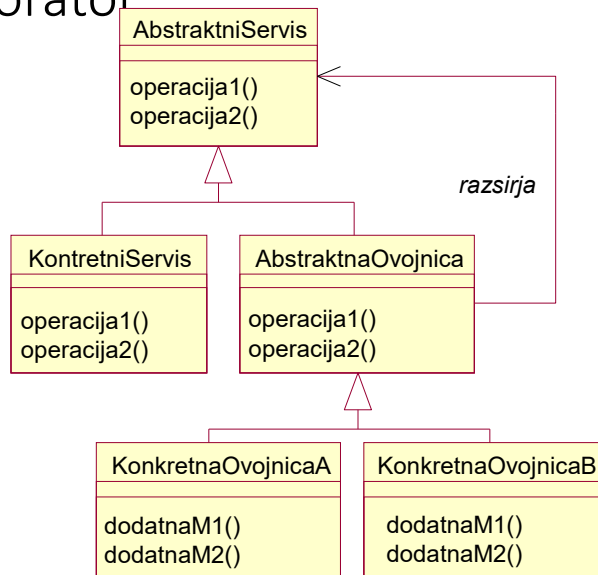
30



```
FileReader fr = new FileReader("rezultati.txt");
BufferedReader vhod = new BufferedReader(fr);
String vrstica;
while ((vrstica = vhod.readLine()) != null)
    System.out.println(vrstica);
```

31

Vzorec Dekorator



32

Vzorec “Dekorator”

- Npr. da potrebujem funkcionalnost za
 - odpiranje zbirke,
 - branje celih števil in
 - “vračanje”, če potrebno.

```

FileReader fr = new FileReader ("abc.txt");
BufferedReader br = new BufferedReader(fr);
int i = br.read();
PushbackReader pr = new PushbackReader (br);
pr.unread(i);

```

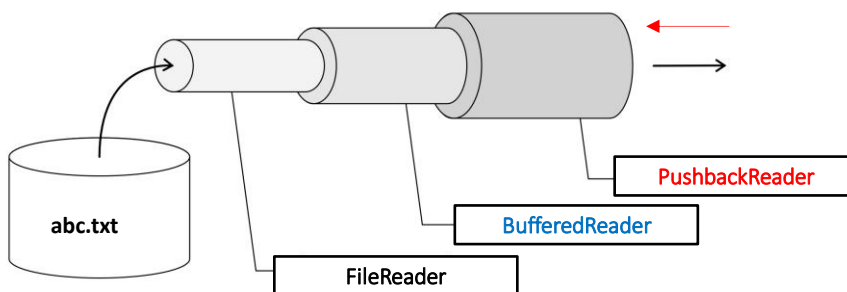
namesto dedovanja in uporabe podrazredov šele v času izvajanja dodajamo objekte, ki implementirajo dodatno funkcionalnost

33

```

FileReader fr = new FileReader ("abc.txt");
BufferedReader br = new BufferedReader(fr);
PushbackReader pr = new PushbackReader (br);

```



34

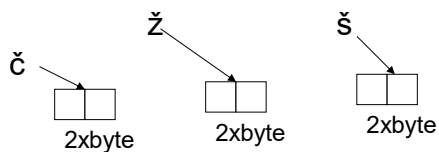
InputStreamReader, OutputStreamReader

- *InputStreamReader* implementira vhodni tok znakov, ki bere zloge iz vhodnega toka zlogov, ki jih pretvori glede na ustrezeni način kodiranja
- *OutputStreamWriter* implementira izhodni tok znakov, ki tok znakov pretvori v zloge (bytes) glede na ustrezeni način kodiranja in jih zapiše v zlogovni tok

35

Kodne tabele

- Kako je z obdelavo datotek v različnih kodnih tabelah?
 - Windows standard (Cp1250)
 - Dos standard (Cp852)
 - Prenosljiva Unicode oblika (UTF-8)



36

UTF-8

Description [\[edit\]](#)

The design of UTF-8 can be seen in this table of the scheme as originally proposed by Dave Prosser and subsequently modified by Ken Thompson (the `x` characters are replaced by the bits of the code point):

Bits of code point	First code point	Last code point	Bytes in sequence	Byte 1	Byte 2	Byte 3	Byte 4	Byte 5	Byte 6
7	U+0000	U+007F	1	0xxxxxxx					
11	U+0080	U+07FF	2	110xxxxx	10xxxxxx				
16	U+0800	U+FFFF	3	1110xxxx	10xxxxxx	10xxxxxx			
21	U+10000	U+1FFFFF	4	11110xxx	10xxxxxx	10xxxxxx	10xxxxxx		
26	U+200000	U+3FFFFFF	5	111110xx	10xxxxxx	10xxxxxx	10xxxxxx	10xxxxxx	
31	U+4000000	U+7FFFFFFF	6	1111110x	10xxxxxx	10xxxxxx	10xxxxxx	10xxxxxx	10xxxxxx

The original specification covered numbers up to 31 bits (the original limit of the [Universal Character Set](#)). In November 2003, UTF-8 was restricted by [RFC 3629](#) to end at U+`10FFFF`, in order to match the constraints of the [UTF-16](#) character encoding. This removed all 5- and 6-byte sequences, and about half of the 4-byte sequences.

The salient features of this scheme are as follows:

- *Backward compatibility*: One-byte codes are used only for the ASCII values 0 through 127. In this case the UTF-8 code has the same value as the ASCII code. The high-order bit of these codes is always 0. This means that UTF-8 can be used for parsers expecting 8-bit [extended ASCII](#) even if they are not designed for UTF-8.

37

Kako določimo kodno tabelo

- Java obdeluje interno vse nize v *Unicode* naboru (2 zloga).
- Pri branju iz zlogovnega niza (`Stream`) lahko navedemo kodno tabelo, v kateri se znaki v toku nahajajo. S tem zagotovimo pravilno pretvorbo v *Unicode* znake.
- Nekatere kodne tabele zahtevajo več zlogov za posamezen znak!!!

38

Določanje kodne tabele

- Določanje kodne tabele pri izhodnih tokovih:

```
FileOutputStream fos1 = new FileOutputStream("datoteka.txt");
OutputStreamWriter w1 = new OutputStreamWriter( fos1,"Cp852");
```

- Določanje kodne tabele pri vhodnih tokovih:

```
FileInputStream fis1 = new FileInputStream("datoteka.txt");
InputStreamReader w1 = new InputStreamReader( fis1,"Cp852");
```

39

	_0	_1	_2	_3	_4	_5	_6	_7	_8	_9	_A	_B	_C	_D	_E	_F
0_	WUL	QON	STX	ETX	EOT	EMQ	ACK	REL	RD	RT	LF	VT	FF	CR	SD	SI
9	0000	0001	0002	0003	0004	0005	0006	0007	0008	0009	000A	000B	000C	000D	000E	000F
1_	DLK	DC1	DC2	DC3	DC4	NAK	SYN	ETB	CAN	EM	SUB	ESC	FS	GS	RS	US
16	0010	0011	0012	0013	0014	0015	0016	0017	0018	0019	001A	001B	001C	001D	001E	001F
2_	SP	!	"	#	\$	%	&	'	(	)	*	+	,	-	.	/
32	0020	0021	0022	0023	0024	0025	0026	0027	0028	0029	002A	002B	002C	002D	002E	002F
3_	0	1	2	3	4	5	6	7	8	9	:	;	<	=	>	?
48	0030	0031	0032	0033	0034	0035	0036	0037	0038	0039	003A	003B	003C	003D	003E	003F
4_	@	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O
64	0040	0041	0042	0043	0044	0045	0046	0047	0048	0049	004A	004B	004C	004D	004E	004F
5_	P	Q	R	S	T	U	V	W	X	Y	Z	[	\	]	^	_
80	0050	0051	0052	0053	0054	0055	0056	0057	0058	0059	005A	005B	005C	005D	005E	005F
6_	`	a	b	c	d	e	f	g	h	i	j	k	l	m	n	o
96	0060	0061	0062	0063	0064	0065	0066	0067	0068	0069	006A	006B	006C	006D	006E	006F
7_	p	q	r	s	t	u	v	w	x	y	z	{		}	~	DEL
112	0070	0071	0072	0073	0074	0075	0076	0077	0078	0079	007A	007B	007C	007D	007E	007F
8_	€		*		...	†	‡		‰	§	€	§	†	‡	€	€
128	0080		203A		203B	203C	203D	203E	203F							
9_		*	*	*	*	*	*	*	TM	§	›	§	›	§	›	§
144	2030	2031	2032	2033	2034	2035	2036	2037	2038	2039	203A	203B	203C	203D	203E	203F
10_	BRBP	*	*	L	Q	A	;	§	*	Q	§	*	§	Q	§	Q
160	00A0	00A1	00A2	00A3	00A4	00A5	00A6	00A7	00A8	00A9	00AA	00AB	00AC	00AD	00AE	00AF
11_	*	*	*	*	*	*	*	*	*	*	*	*	*	*	*	*
176	00B0	00B1	00B2	00B3	00B4	00B5	00B6	00B7	00B8	00B9	00BA	00BB	00BC	00BD	00BE	00BF
12_	€	€	€	€	€	€	€	€	€	€	€	€	€	€	€	€
192	00C0	00C1	00C2	00C3	00C4	00C5	00C6	00C7	00C8	00C9	00CA	00CB	00CC	00CD	00CE	00CF
13_	D	N	N	O	O	O	O	x	R	U	U	U	U	U	Y	T
208	00D0	00D1	00D2	00D3	00D4	00D5	00D6	00D7	00D8	00D9	00DA	00DB	00DC	00DD	00DE	00DF
14_	f	a	a	a	a	a	i	e	e	e	e	e	e	e	i	i
224	00E0	00E1	00E2	00E3	00E4	00E5	00E6	00E7	00E8	00E9	00EA	00EB	00EC	00ED	00EE	00EF
15_	d	a	a	o	o	o	o	+	f	a	u	u	u	u	y	t
240	00F0	00F1	00F2	00F3	00F4	00F5	00F6	00F7	00F8	00F9	00FA	00FB	00FC	00FD	00FE	00FF

Letter Number Punctuation Symbol Other Undefined

The following table shows code page 852^{[2][10]}. Each character is shown with its equivalent Unicode code point. Only the second half of the table (128–255) is shown, the

	_0	_1	_2	_3	_4	_5	_6	_7	_8	_9	_A	_B	_C	_D	_E	_F
128	00C7	00FC	0089	0082	0084	014F	0107	0087	0142	008B	015D	0153	008B	0179	00C8	0156
9_	É	Ê	Ë	Ô	Õ	Ö	Ç	È	É	Ê	Ë	Ô	Õ	Ö	Ç	È
144	0009	0139	013A	00F4	00F6	013D	013B	013A	013B	00D6	00C0	0143	0145	0141	00D7	013D
160	00E1	00ED	00F3	00FA	0104	0105	017D	017B	0133	0139	00AC	017A	0100	015B	00AD	00BB
176	2591	2592	2593	2502	2524	00C1	00C2	013B	013D	2543	2551	2557	255D	0179	017D	2510
192	2514	2534	253C	251C	2500	253C	0102	0103	255A	2554	2549	2546	2540	2550	256C	00B4
208	00D1	00D2	00D3	00D4	00D5	00D6	00D7	00D8	00D9	00DA	00DB	00DC	00DD	00DE	00DF	00E0
224	00E1	00E2	00E3	00E4	00E5	00E6	00E7	00E8	00E9	00EA	00EB	00EC	00ED	00EE	00EF	00F0
240	00F1	00F2	00F3	00F4	00F5	00F6	00F7	00F8	00F9	00FA	00FB	00FC	00FD	00FE	00FF	0000

40

Serializacija

Shranjevanje objektov

41

Serializacija

- objekte lahko serializiramo, pretvorimo v tok zlogov, ki ga shranimo v zbirko, pošljemo preko omrežja, kjer ga lahko popolnoma rekonstruiramo na osnovi njegovega serializiranega “zlogovnega stream-a” (binarnega toka)
- aplikacija lahko s serializacijo objektov torej shrani stanje

42

Pomen serializacije

- dodana je bila v verziji 1.1 za podporo
 - proženja oddaljenih metod (RMI Remote Method Invocation)
prenesti je potrebno argumente in rezultate - vrnjene vrednosti
 - Java zrnom (Java Beans)
informacije o stanju zrna običajno nastavimo in določimo v času načrtovanja, ob zagonu programa jih je potrebno ponovno vzpostaviti (v ta namen uporabimo serializacijo)

43

Standardne knjižnice

- veliko razredov iz standardnih knjižnic implementira vmesnik **Serializable**, vključno z vsemi "ovojnicami" osnovnih tipov, kolekcije ...

44

Serializacija

```
ObjectOutputStream oos =  
    new ObjectOutputStream (  
        new GZIPOutputStream(  
            new FileOutputStream(  
                String.valueOf(args[0]) +  
                ".ser"))));  
oos.writeObject(nekObjekt);
```

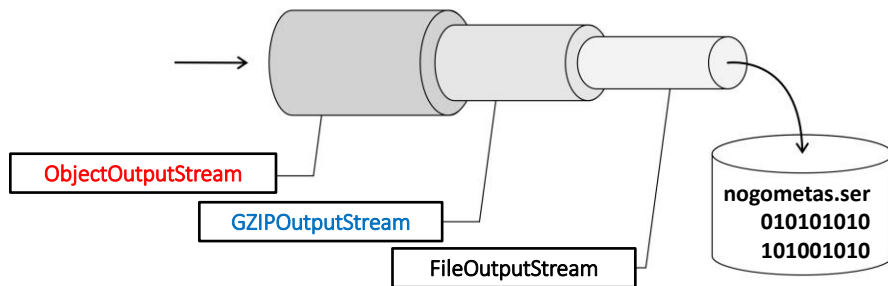
45

Kako?

- kreiramo nek objekt tipa **OutputStream** in ga ovijemo z objektom **ObjectOutputStream**, nakar prožimo **writeObject()**
- kreiramo nek objekt tipa **InputStream** in ga ovijemo z objektom **ObjectInputStream**, nakar prožimo **readObject**, dobimo **Object**, ki ga s pomočjo "downcast"-a pretvorimo v ustrezni tip

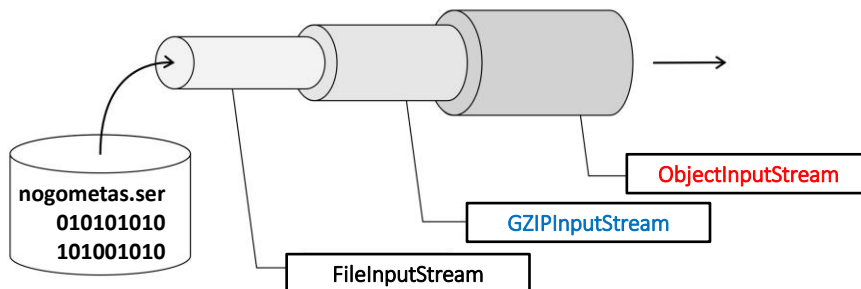
46

serializacija



47

deserializacija



48

Deserializacija

```
import java.io.*;
import java.util.zip.*;

try {
    ObjectInputStream ois =
        new ObjectInputStream (
            new GZipInputStream (
                new FileInputStream("ime.ser")));
    nekaj = (TipNekaj) ois.readObject();
}
catch (Exception e) {
    System.out.println("Tezave "+e.toString());
}
}
```

49

Torej

- uporaba **writeObject** metode razreda **ObjectOutputStream**, ki zagotavlja metode za serializacijo Java primitiv, npr. **writeInt**, **writeChar**, **writeLong** itd.
- obratni postopek **readObject** metoda razreda **ObjectInputStream**
- če želimo zgoščeno obliko uporabimo paket **java.util.zip**
- atributov - objektov, ki ne podpirajo vmesnik **Serializable**, ne moremo serializirati
- s **transient** označimo prehodnost

50